

Duration: 2 Hours

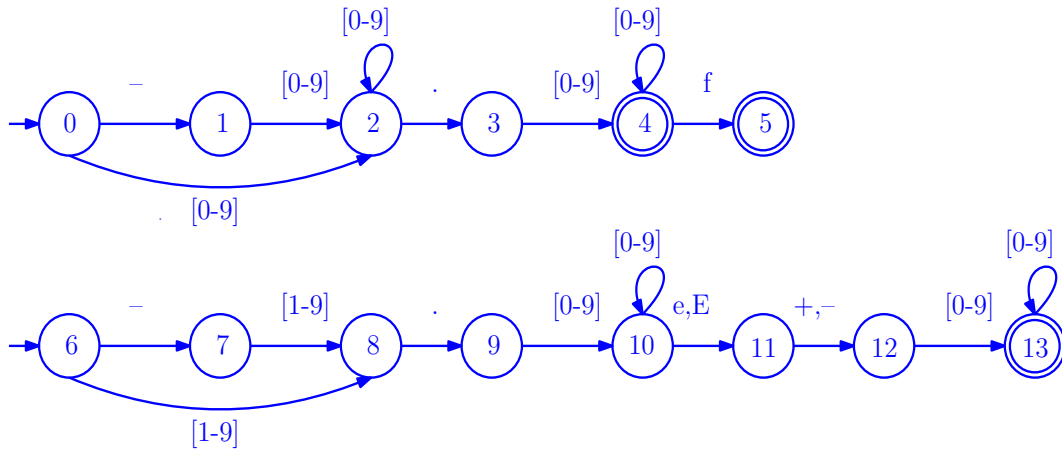
Full Marks: 60

Number of pages: 1

- Attempt all questions.
- All parts of a question should be answered together.
- Grammars have upper case letters as non-terminals, rest of the symbols are terminals.

Q 1. According to C programming language the real numbers (such as 55.33) can be represented as a floating point constant (55.33f), double precision constant (55.33) and in scientific notation (5.533E+10). Construct a deterministic finite automaton that will recognize the real number constants in C programs using proper design methodology. [12]

Solution: The required deterministic finite automata for floating point constants in float, double and scientific notations are given as follows.



- Q 2. (a) Design a recursive descent parser that will accept palindromes of the form wxw^R where string $w \in \Sigma^*$, $\Sigma = \{a, b, c\}$, x is a terminal representing the middle symbol, and w^R is reverse of w . [8]
 Solution: The required context free grammar rules are $S \rightarrow aSa|bSb|cSc|x$. The recursive descent parser for this grammar is given as follows.

```

S()
{
    if(*input_ptr == 'x') // S → x
    {
        input_ptr++;
        return true;
    }
    else
        if(*input_ptr == 'a') // S → aSa
        {
            input_ptr++;
            if(S() == true)
            {
                if(*input_ptr == 'a')
                {
                    input_ptr++;
                    return true;
                }
            }
            return false;
        }
    else
        if(*input_ptr == 'b') // S → bSb
        {
            input_ptr++;
            if(S() == true)
            {
                if(*input_ptr == 'b')
                {
                    input_ptr++;
                    return true;
                }
            }
            return false;
        }
    else
        if(*input_ptr == 'c') // S → cSc
        {
            input_ptr++;
            if(S() == true)
            {
                if(*input_ptr == 'c')
                {
                    input_ptr++;
                    return true;
                }
            }
            return false;
        }
    else
        return false;
}

```

- (b) Use the parser to derive the input string “bacxcab\$”. Show the stack content and rule applied for each step. [4]

Solution:

Table 1: Derivation of “bacxcab\$”

Stack	Input	Action	Rule
\$	<u>b</u> acxcab\$	Call $S()$	
\$S	<u>b</u> acxcab\$		
\$SbSb	<u>b</u> acxcab\$	Match b , advance $input_ptr$, call $S()$	$S \rightarrow bSb$
\$SbSaSa	<u>b</u> acxcab\$	Match a , advance $input_ptr$, call $S()$	$S \rightarrow aSa$
\$SbSaScSc	<u>b</u> acxcab\$	Match c , advance $input_ptr$, call $S()$	$S \rightarrow cSc$
\$SbSaScSSx	ba <u>x</u> cab\$	Match x , advance $input_ptr$, $S()$ returns $true$	$S \rightarrow x$
\$SbSaScS	ba <u>c</u> cab\$	Match c , advance $input_ptr$, $S()$ returns $true$	
\$SbSaS	ba <u>x</u> cab\$	Match a , advance $input_ptr$, $S()$ returns $true$	
\$SbS	ba <u>x</u> cab\$	Match b , advance $input_ptr$, $S()$ returns $true$	
\$S	ba <u>x</u> cab\$	$S()$ returns $true$	
\$	ba <u>x</u> cab\$	Input string matched	

- Q 3. For the following grammar with S as starting symbol, find FIRST and FOLLOW sets of each of the non-terminals. $S \rightarrow aAB|bA|\epsilon$, $A \rightarrow aAb|\epsilon$, $B \rightarrow bB|c$. [12]

Solution: $FIRST(S) = FIRST(aAB) \cup FIRST(bA) \cup FIRST(\epsilon) = \{a, b, \epsilon\}$
 $FIRST(A) = FIRST(aAb) \cup FIRST(\epsilon) = \{a, \epsilon\}$
 $FIRST(B) = FIRST(bB) \cup FIRST(c) = \{b, c\}$
 $FOLLOW(S) = \{\$ \}$
 $FOLLOW(A) = FIRST(B) \cup FOLLOW(S) = \{b, c, \$ \}$
 $FOLLOW(B) = FOLLOW(S) = \{\$ \}$

- Q 4. Consider a grammar G with starting symbol S .
The rule set of G is $\{S \rightarrow abSa|aaAb|b, A \rightarrow baAb|b\}$.

- (a) Verify whether $G \in LL(1)$. Use the definition of $LL(1)$ grammar. [4]

Solution: $FIRST(S) = FIRST(abSa) \cup FIRST(aaAb) \cup FIRST(b) = \{a, b\}$
 $FIRST(A) = FIRST(baAb) \cup FIRST(b) = \{b\}$
For rule $S \rightarrow abSa|aaAb|b$, $FIRST(abSa) \cap FIRST(aaAb) \neq \emptyset$
For rule $A \rightarrow baAb|b$, $FIRST(baAb) \cap FIRST(b) \neq \emptyset$
Hence $G \notin LL(1)$.

- (b) Construct a predictive parser table for G . [8]

Table 2: Predictive parsing table for G

	a	b	$\$$
S	$S \rightarrow abSa, S \rightarrow aaAb$	$S \rightarrow b$	
A		$A \rightarrow baAb, A \rightarrow b$	

Q 5. Given a grammar with rules $S \rightarrow (S)|id$

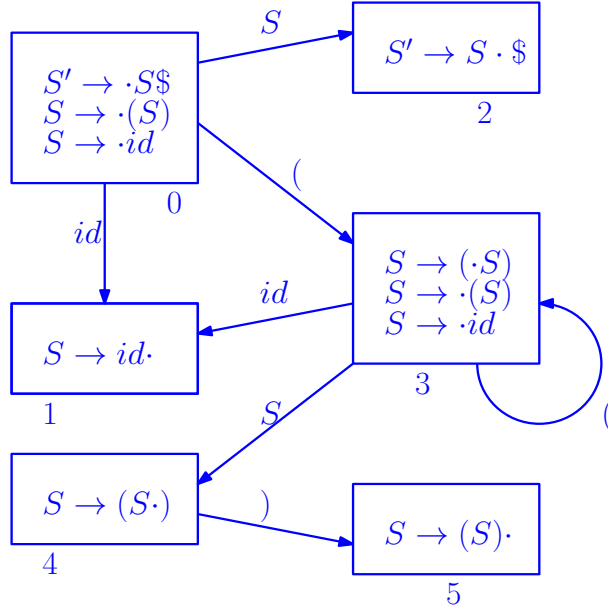
(a) Show that the grammar is LR(0).

[4]

Soluion: The augmented grammar for the given grammar is

Rule 0: $S' \rightarrow S\$$, Rule 1: $S \rightarrow (S)$, Rule 2: $S \rightarrow id$

The DFA of LR(0) set of items using GOTO and CLOSURE is given as follows.



All the set of items are free of shift-reduce conflicts. Hence the given grammar is LR(0).

(b) Construct a parsing table for the grammar. Specify the entries for all possible errors.

[6]

The errors with corresponding error codes are listed as follows

E0:) missing for a (

E1: (missing for a)

E2: id missing in between (and)

E3: (cannot be after id

E4: (cannot be after)

E5: id cannot be after id

E6: Nothing before \$

Solution: The parsing table for the given LR(0) grammar is as follows.

State	Action				Goto
	(	)	\$	id	S
0	s3	E1/E4	E6	s1	2
1	r2	r2	r2	r2	
2	E3	E1	accept	E5	
3	s3	E2	E2	s1	4
4	E0	s5	E0	E0	
5	r1	r1	r1	r1	

Q 6. Derive the input string " id)\$". Show the stack content and action for each step.

[2]

Solution:

Table 3: Derivation of “ id ”\$

Stack	Input	Action	Rule
\$0	<u>id</u> \$	Shift id , goto state 1	
\$0 id 1	id \$	Reduce by rule 2, goto state 2	$S \rightarrow id$
\$0 S 2	id \$	Error E1: (missing for a)	

*****All the best*****